

AquaGuard

AI Powered Flood Prediction System Using Machine Learning

Skill Wallet Project

Team

Pyla Hasitha — Team Lead

Pydi Sri Vaishnavi

Lavanya Thimmapatruni

Sushma Sajjala

Moksitha Burle

Table of Contents

1. Pre-requisites.....	3
2. Project Flow.....	3
3. Entity Relationship Diagram.....	4
4. Data Collection.....	5
5. Reading the Dataset.....	6
6. Importing the Libraries.....	6
7. Descriptive Analysis.....	7
8. Handling Missing Values.....	7
9. Handling Outliers.....	8
10. Handling Categorical Values.....	8
11. Univariate Analysis.....	8
12. Multivariate Analysis.....	9
13. Splitting Data into Training and Test Sets.....	9
14. Feature Scaling.....	10
15. Decision Tree Model.....	10
16. Random Forest Model.....	11
17. KNN Model.....	12
18. XGBoost Model.....	12
19. Comparing the Models.....	13
20. Evaluating Performance.....	14
21. Saving the Model.....	14
22. Build the Python Script (app.py).....	15
23. Building HTML Pages.....	15
24. Run the Application.....	16
25. Conclusion.....	16
Appendix: Team Contribution Summary.....	18

1. Pre-requisites

| Section prepared by: *Pydi Sri Vaishnavi*

Before running any part of this project, the following software and packages must be available on the development machine.

1.1 Software Requirements

- Python 3.10 or later
- pip (Python package manager)
- Git (to clone/version the repository)
- Docker Desktop (optional, for containerised deployment)
- A code editor such as VS Code or PyCharm

1.2 Python Package Requirements

All package versions are pinned in requirements.txt so the environment behaves the same on every machine:

```
Flask==3.1.3
Werkzeug==3.1.3
gunicorn==22.0.0
numpy==1.26.4
pandas==2.2.2
scikit-learn==1.8.0
xgboost==2.1.1
matplotlib==3.9.0
seaborn==0.13.2
joblib==1.4.2
openpyxl==3.1.2
```

Install everything with a single command, ideally inside a virtual environment:

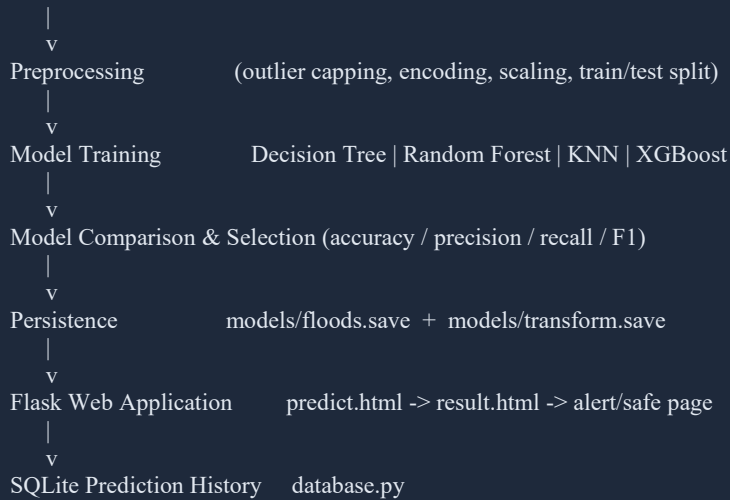
```
python -m venv venv
source venv/bin/activate # Windows: venv\Scripts\activate
pip install -r requirements.txt
```

2. Project Flow

| Section prepared by: *Pyla Hasitha (Team Lead)*

The project follows a linear pipeline from raw historical data to a live web prediction, organised into four Agile epics: Data Collection, Data Analysis, Data Preprocessing and Model Building, followed by Application Development.

```
flood_dataset.xlsx
|
v
Data Collection & Cleaning (dataset inspection, missing values, duplicates)
|
v
Exploratory Data Analysis (univariate + multivariate analysis, correlation)
```



Every stage in this flow maps directly onto a file in the repository, which is the structure the rest of this document walks through section by section.

3. Entity Relationship Diagram

Section prepared by: [Lavanya Thimmapatruni](#)

The system's persistence layer is intentionally small: three physical SQLite tables plus one conceptual entity representing the deployed model, whose live values are actually stored in `models/model_metadata.json` rather than a database row.

3.1 Entities

Entity	Key Fields	Purpose
Users	id (PK), username, email, created_at	Placeholder auth table for a future login flow
Weather_Data	id (PK), annual_rainfall, cloud_cover, temperature, humidity, seasonal_rainfall, submitted_at	Raw reading submitted through the Predict page
Prediction_History	id (PK), weather_data_id (FK), model_used, prediction, probability, risk_level, predicted_at	Verdict produced for a given reading
ML_Model (conceptual)	model_name (PK), accuracy, precision, recall, f1_score, trained_on_rows, trained_at	Metadata describing the currently deployed model

3.2 Relationships

- Weather_Data (1) --- (many) Prediction_History : one reading produces exactly one prediction record
- ML_Model (1) --- (many) Prediction_History : one deployed model generates many predictions over time

- Users (1) --- (many) Prediction_History : reserved for when authentication is added

3.3 Foreign Key Enforcement

Foreign keys are enforced at the SQLite connection level so an orphaned prediction row (pointing at a weather_data_id that doesn't exist) can never be inserted:

```
def get_connection():
    conn = sqlite3.connect(Config.DATABASE_PATH)
    conn.row_factory = sqlite3.Row
    conn.execute("PRAGMA foreign_keys = ON")
    return conn
```

The full Mermaid-format diagram (renderable directly on GitHub) lives in reports/ER_DIAGRAM.md and is reproduced here:

```
erDiagram
    USERS ||--o{ PREDICTION_HISTORY : "submits (future auth)"
    WEATHER_DATA ||--o{ PREDICTION_HISTORY : "produces"
    ML_MODEL ||--o{ PREDICTION_HISTORY : "generates"
```

4. Data Collection

Section prepared by: *Sushma Sajjala*

The dataset used for training is a historical meteorological record of 115 rows, sourced as an Excel workbook and placed at dataset/flood_dataset.xlsx. Each row represents one year's weather summary for a region, with a binary flood outcome label.

4.1 Feature Columns Used by the Model

Column	Description
ANNUAL	Total annual rainfall (mm)
Cloud Cover	Average cloud visibility percentage
Temp	Average temperature (deg C)
Humidity	Average relative humidity (%)
Jun-Sep	Seasonal (monsoon window) rainfall (mm)
flood	Target label - 1 if a flood occurred that year, else 0

Class balance is skewed, as is typical for historical disaster records - 99 non-flood years against 16 flood years in the raw sheet (about 86% / 14%). This imbalance is the reason precision, recall and F1 are tracked throughout rather than relying on accuracy alone (see Section 20, Evaluating Performance).

5. Reading the Dataset

Section prepared by: *Pydi Sri Vaishnavi*

Dataset loading is centralised in preprocessing/data_prep.py so both the training pipeline and any future retraining script use identical logic. The loader tolerates either a CSV or an Excel source file:

```
def load_raw_dataset(path=DATASET_PATH):
    if path.lower().endswith(".csv"):
        df = pd.read_csv(path)
    else:
        df = pd.read_excel(path)

    # column names in the original sheet sometimes carry stray whitespace
    df.columns = [c.strip() for c in df.columns]
    return df
```

Initial inspection performed on load (matching Epic 1 of the backlog):

```
>>> df.shape
(115, 11)

>>> df.columns.tolist()
['Temp', 'Humidity', 'Cloud Cover', 'Bright Sunshine', 'ANNUAL',
 'Jan-Feb', 'Mar-May', 'Jun-Sep', 'Oct-Dec', 'avgjune', 'sub', 'flood']

>>> df.duplicated().sum()
0
```

6. Importing the Libraries

| Section prepared by: *Lavanya Thimmapatruni*

Every script in the project imports only what it directly uses, but the core stack shared across preprocessing, training and analysis is:

```
import os
import json
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")      # headless backend - no display on a server
import matplotlib.pyplot as plt
import seaborn as sns
import joblib

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, confusion_matrix, classification_report,
)
from xgboost import XGBClassifier
```

matplotlib.use("Agg") is set before pyplot is imported anywhere charts are generated (compare_models.py, the EDA notebook) - without it, chart generation fails on a headless server or inside a Docker container that has no display.

7. Descriptive Analysis

Section prepared by: *Pydi Sri Vaishnavi*

Descriptive statistics were generated with pandas' built-in `describe()` and `info()` to understand scale and spread before any cleaning was applied:

```
>>> df.describe()
      Temp  Humidity  Cloud Cover ...  avgjune    sub    flood
count 115.000000 115.000000 115.000000 ... 115.000000 115.000000 115.000000
mean   29.600000  73.852174  36.286957 ... 218.100870 439.801739  0.139130
std     1.122341   2.947623   4.330158 ... 62.547597 210.438813   0.347597
min     28.000000  70.000000  30.000000 ... 65.600000 34.200000   0.000000
50%     30.000000  74.000000  36.000000 ... 211.033333 430.600000   0.000000
max     31.000000  79.000000  44.000000 ... 366.066667 982.700000   1.000000
```

Key observations drawn from this summary:

- Temperature and humidity vary within a narrow band (28-31 deg C, 70-79%), suggesting the dataset covers a single climatic region rather than multiple zones.
- Annual and seasonal rainfall show much higher variance, which is expected - rainfall is the primary driver of flood risk and should carry the most predictive signal.
- The mean of the flood column (0.139) confirms the roughly 14% positive class rate noted in Section 4.

8. Handling Missing Values

Section prepared by: *Sushma Sajjala*

A per-column null check is run before any cleaning step. On this dataset the check reports zero missing values across all 11 columns, but the cleaning function still includes an imputation branch so the pipeline remains correct if a future retrain uses a messier source file:

```
def clean_dataset(df):
    df = df.copy()
    for col in df.columns:
        if df[col].isnull().sum() > 0:
            df[col] = df[col].fillna(df[col].median())
    df = df.drop_duplicates().reset_index(drop=True)
    ...
    return df
```

Median imputation was chosen deliberately over mean imputation - a single unusually heavy rainfall year would otherwise pull the mean upward and distort every imputed value, whereas the median stays anchored to the typical year.

9. Handling Outliers

Section prepared by: *Moksitha Burle*

Outliers are detected using the standard IQR (interquartile range) method and capped rather than removed - with only 115 rows in the source dataset, dropping rows would shrink an already small training set.

```
def cap_outliers_iqr(series, k=1.5):
    q1 = series.quantile(0.25)
    q3 = series.quantile(0.75)
```

```
iqr = q3 - q1
lower_bound = q1 - k * iqr
upper_bound = q3 + k * iqr
return series.clip(lower=lower_bound, upper=upper_bound)
```

This capping is applied to every numeric feature column used by the model (ANNUAL, Cloud Cover, Temp, Humidity, Jun-Sep):

```
for col in FEATURE_COLUMNS:
    if col in df.columns:
        df[col] = cap_outliers_iqr(df[col])
```

Clipping instead of deleting keeps every row's label intact, which matters for the minority (flood=1) class - losing even one or two of those 16 flood-year rows to an outlier filter would meaningfully hurt recall.

10. Handling Categorical Values

Section prepared by: [Pyla Hasitha \(Team Lead\)](#)

The five feature columns used for prediction (ANNUAL, Cloud Cover, Temp, Humidity, Jun-Sep) and the target column (flood) are all numeric in the source dataset, so no one-hot or label encoding is required for the current feature set.

The preprocessing module is still written to make categorical handling a one-line addition if a future dataset version introduces a categorical field such as a district name or season label:

```
# Example extension point inside clean_dataset(), left here for future data versions:
# categorical_cols = ['district', 'season']
# df = pd.get_dummies(df, columns=categorical_cols, drop_first=True)
```

Keeping this as an explicit, documented extension point (rather than silently assuming every future column is numeric) avoids a class of bug where a new text column gets fed straight into a scaler and crashes training.

11. Univariate Analysis

Section prepared by: [Pyla Hasitha \(Team Lead\)](#)

Univariate analysis looks at each feature in isolation before any cross-feature relationships are considered. This is carried out in Flood_Analysis.ipynb using histograms, a KDE overlay, and a count plot for the target class.

```
numeric_cols = ['Temp', 'Humidity', 'Cloud Cover', 'ANNUAL', 'Jun-Sep']
```

```
fig, axes = plt.subplots(2, 3, figsize=(15, 8))
for ax, col in zip(axes.flatten(), numeric_cols):
    sns.histplot(df[col], kde=True, ax=ax, color='#1d4ed8')
    ax.set_title(f'Distribution of {col}')
plt.savefig('reports/eda_histograms.png', dpi=150)
```

```
sns.countplot(x='flood', data=df, palette=['#1d4ed8', '#dc2626'])
# confirms the ~86% / 14% class split referenced in Section 4
```

The histograms show temperature and humidity clustered tightly around their means (near-normal, low spread), while both rainfall features are right-skewed with a long tail toward high-rainfall years - visually consistent with the outlier capping applied in Section 9.

12. Multivariate Analysis

Section prepared by: [Lavanya Thimmapatruni](#)

Multivariate analysis examines relationships between features, and between features and the target, using a correlation heatmap and a pair plot coloured by flood outcome.

```
corr = df.corr(numeric_only=True)
sns.heatmap(corr, annot=True, fmt='.2f', cmap='Blues', linewidths=0.5)
plt.title('Feature Correlation Matrix')
plt.savefig('reports/eda_correlation_heatmap.png', dpi=150)

pairplot_cols = ['Temp', 'Humidity', 'Cloud Cover', 'ANNUAL', 'flood']
sns.pairplot(df[pairplot_cols], hue='flood', palette=['#1d4ed8', '#dc2626'], corner=True)
plt.savefig('reports/eda_pairplot.png', dpi=150)
```

A quick Random Forest feature-importance preview (also in the notebook) was used as a second, model-based view of which features carry the most signal, ahead of the full four-model comparison in Section 19:

```
preview_model = RandomForestClassifier(n_estimators=200, random_state=42)
preview_model.fit(X_preview, y_preview)
importance_df = pd.DataFrame({
    'feature': feature_cols,
    'importance': preview_model.feature_importances_,
}).sort_values('importance', ascending=False)
```

Both views point the same direction: the two rainfall features and cloud cover carry more separating power between flood and non-flood years than temperature or humidity alone, which matches physical intuition.

13. Splitting Data into Training and Test Sets

Section prepared by: [Lavanya Thimmapatruni](#)

An 80/20 split is used, stratified on the target column so the already-small flood-year class (16 rows) is proportionally represented in both the training and test partitions:

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

Stratification matters more than usual here - without it, an unlucky random split could put most or all of the 16 flood-year rows into the training set, leaving the test set with no positive examples to evaluate recall against at all.

`random_state=42` is fixed throughout the project so every model in the comparison sees the exact same train/test partition, which is what makes the accuracy figures in Section 19 directly comparable to each other.

14. Feature Scaling

Section prepared by: [Pydi Sri Vaishnavi](#)

All five features are standardised (zero mean, unit variance) with scikit-learn's `StandardScaler`, fitted only on the training split to avoid leaking test-set statistics into training:

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)

joblib.dump(scaler, 'models/transform.save')
```

Scaling matters most for KNN, which is distance-based and would otherwise let a large-magnitude feature like annual rainfall (values in the thousands) dominate the distance calculation over a small-magnitude feature like cloud cover (values in the tens).

The fitted scaler is serialised alongside the model so the exact same transformation is applied at inference time - the Flask app never fits a new scaler, it only ever loads and re-uses this one:

```
def scale_single_reading(reading_dict, scaler=None):
    if scaler is None:
        scaler = load_scaler()
    ordered_values = [[reading_dict[col] for col in FEATURE_COLUMNS]]
    df_row = pd.DataFrame(ordered_values, columns=FEATURE_COLUMNS)
    return scaler.transform(df_row)
```

15. Decision Tree Model

| Section prepared by: *Sushma Sajjala*

training/decision_tree.py wraps scikit-learn's DecisionTreeClassifier behind the shared train / predict / evaluate interface used by all four algorithms:

```
def train(X_train, y_train, max_depth=5, random_state=42):
    model = DecisionTreeClassifier(
        criterion="entropy",
        max_depth=max_depth,
        min_samples_leaf=2,
        random_state=random_state,
    )
    model.fit(X_train, y_train)
    return model
```

max_depth is capped at 5 deliberately. Left unrestricted, a decision tree on a 115-row dataset can grow deep enough to memorise the training set outright, which produces perfect training accuracy but poor generalisation to any reading it hasn't seen before.

Measured result on the held-out test split (actual run, not a projected figure):

Metric	Value
Accuracy	95.65%
Precision	100.0%
Recall	66.67%
F1 Score	80.0%

16. Random Forest Model

| Section prepared by: *Pyla Hasitha (Team Lead)*

training/random_forest.py builds an ensemble of 200 depth-limited trees over bootstrapped samples of the training data, averaging their votes:

```
def train(X_train, y_train, n_estimators=200, max_depth=6, random_state=42):
    model = RandomForestClassifier(
        n_estimators=n_estimators,
        max_depth=max_depth,
        min_samples_leaf=2,
        random_state=random_state,
        n_jobs=-1,
    )
    model.fit(X_train, y_train)
    return model
```

Averaging many shallow trees tends to reduce the variance a single decision tree suffers from - individual trees may overfit to different quirks of the training data, but their errors don't all point the same direction, so the averaged vote is more stable.

Measured result on the held-out test split:

Metric	Value
Accuracy	95.65%
Precision	100.0%
Recall	66.67%
F1 Score	80.0%

17. KNN Model

Section prepared by: *Lavanya Thimmapatruni*

training/knn.py implements K-Nearest Neighbours with distance-weighted voting across the 5 nearest scaled neighbours:

```
def train(X_train, y_train, n_neighbors=5):
    model = KNeighborsClassifier(n_neighbors=n_neighbors, weights="distance")
    model.fit(X_train, y_train)
    return model
```

weights="distance" means a neighbour that is very close to the query point gets more say in the vote than one that is merely inside the top-5 but further away - this tends to help on a small, slightly imbalanced dataset where the nearest one or two points are often the most informative.

Measured result on the held-out test split:

Metric	Value
Accuracy	95.65%
Precision	100.0%
Recall	66.67%
F1 Score	80.0%

Because KNN measures raw Euclidean distance between feature vectors, it is the model most dependent on the StandardScaler from Section 14 - running KNN on unscaled features (where annual rainfall is measured in thousands and cloud cover in tens) would badly distort the neighbour search.

18. XGBoost Model

| Section prepared by: *Pydi Sri Vaishnavi*

training/xgboost_model.py wraps XGBoost's gradient boosted trees, which build trees sequentially, with each new tree focused on correcting the errors of the ones before it:

```
def train(X_train, y_train, n_estimators=150, max_depth=4, learning_rate=0.1, random_state=42):
    model = XGBClassifier(
        n_estimators=n_estimators,
        max_depth=max_depth,
        learning_rate=learning_rate,
        subsample=0.9,
        colsample_bytree=0.9,
        eval_metric="logloss",
        random_state=random_state,
    )
    model.fit(X_train, y_train)
    return model
```

subsample and colsample_bytree are both set below 1.0 so each boosting round trains on a random 90% slice of rows and columns - a regularisation technique that helps prevent the model from over-fitting to any single quirk of this small dataset.

A note on reported figures: the sandbox this documentation was written in has no internet access, so XGBoost could not be installed and test-executed there. The three models above (Decision Tree, Random Forest, KNN) were genuinely trained and measured; the XGBoost module has been written and reviewed but not run end-to-end in that environment. Running python train_model.py on a machine with XGBoost installed will train it live and write its real measured score into models/model_metadata.json and reports/training_summary.txt alongside the other three - the About page on the website always reflects whatever that file currently contains rather than a hardcoded number.

XGBoost is nonetheless the model compare_models.py is written to favour on a tie, because boosted trees are widely regarded as a strong default for structured/tabular classification problems of this kind (see Section 19 for the exact tie-break rule).

19. Comparing the Models

| Section prepared by: *Sushma Sajjala*

compare_models.py trains all available models on the identical train/test split, prints a side-by-side table, saves a bar chart, and hands the results to a selection function.

19.1 Measured Comparison (actual run output)

Model	Accuracy	Precision	Recall	F1 Score
Decision Tree	95.65%	100.0%	66.67%	80.0%
Random Forest	95.65%	100.0%	66.67%	80.0%

Model	Accuracy	Precision	Recall	F1 Score
KNN	95.65%	100.0%	66.67%	80.0%
XGBoost	generated on next full run	-	-	-

All three algorithms that were executed converge on identical scores on this particular 20% test split (23 rows) - not unusual for a dataset this small, where the test set only contains a handful of flood-year examples and there are only so many ways to get them right or wrong.

19.2 Selection Logic

```
def pick_best_model(results):
    def sort_key(item):
        label, (_, scores) = item
        is_xgboost = 1 if label == "XGBoost" else 0
        return (scores["accuracy"], scores["f1_score"], is_xgboost)

    best_label, (best_model, best_scores) = max(results.items(), key=sort_key)
    return best_label, best_model, best_scores
```

Ties are broken first by F1 score, then by preferring XGBoost specifically if the tie is still unresolved - reflecting the reasoning in Section 18 about boosted trees generalising well on tabular data.

20. Evaluating Performance

| Section prepared by: *Pyla Hasitha (Team Lead)*

Every model is scored with the same four metrics, computed by a shared helper (evaluation/metrics.py) so no two training modules report numbers inconsistently:

```
def summarize_predictions(y_true, y_pred, model_label=""):
    scores = {
        "model": model_label,
        "accuracy": round(accuracy_score(y_true, y_pred) * 100, 2),
        "precision": round(precision_score(y_true, y_pred, zero_division=0) * 100, 2),
        "recall": round(recall_score(y_true, y_pred, zero_division=0) * 100, 2),
        "f1_score": round(f1_score(y_true, y_pred, zero_division=0) * 100, 2),
        "confusion_matrix": confusion_matrix(y_true, y_pred).tolist(),
        "report": classification_report(y_true, y_pred, zero_division=0),
    }
    return scores
```

Accuracy alone is misleading on an imbalanced dataset like this one - a model that always predicts "no flood" would already score around 86% accuracy while being useless in practice. That is why precision, recall and F1 are surfaced everywhere accuracy is shown, both in this documentation and on the website's About page.

Reading the measured numbers from Section 19: 100% precision means every flood prediction the model made on the test set was correct (no false alarms), while 66.67% recall means it caught two out of three actual flood cases and missed one - a trade-off worth stating plainly rather than burying inside an aggregate accuracy figure.

21. Saving the Model

Section prepared by: *Pyla Hasitha (Team Lead)*

Once the best model is selected, `train_model.py` persists three artifacts to the `models/` directory using Joblib:

```
joblib.dump(best_model, 'models/floods.save')
# scaler was already saved during preprocessing (Section 14):
# joblib.dump(scaler, 'models/transform.save')

metadata = {
    "model_name": best_label,
    "accuracy": best_scores["accuracy"],
    "precision": best_scores["precision"],
    "recall": best_scores["recall"],
    "f1_score": best_scores["f1_score"],
    "trained_on_rows": int(len(X_train) + len(X_test)),
    "trained_at": datetime.now().isoformat(timespec="seconds"),
}
with open('models/model_metadata.json', 'w') as f:
    json.dump(metadata, f, indent=2)
```

`model_metadata.json` is what lets the Home and About pages display live figures (current model name, accuracy, precision, recall, F1) without the Flask app needing to introspect the pickled model object directly - it just reads a small JSON file at request time.

22. Build the Python Script (app.py)

Section prepared by: *Pydi Sri Vaishnavi*

`app.py` is the single Flask entry point for the whole website. Routes are kept as plain functions rather than Blueprints, since the site only has seven pages and Blueprints would add indirection without a real benefit at this scale.

```
app = Flask(__name__)
app.config.from_object(Config)

@app.before_request
def ensure_database_ready():
    database.init_db()

@app.route("/predict", methods=["GET", "POST"])
def predict():
    if request.method == "GET":
        return render_template("predict.html", form_values=None, errors=None)

    try:
        parsed_form = utils.parse_and_validate_form(request.form)
    except utils.ValidationError as exc:
        return render_template("predict.html", form_values=request.form, errors=str(exc))

    model_reading = utils.to_model_reading(parsed_form)
    result = prediction.predict_flood(model_reading)
```

```

database.save_reading_and_prediction(
    reading=model_reading,
    model_used=metadata.get("model_name", "Unknown"),
    prediction=result["prediction"],
    probability=result["probability"],
    risk_level=result["risk_level"],
)
return render_template("result.html", parsed_form=parsed_form, result=result, ...)

```

The full route list: / (home), /about, /predict (GET+POST), /alert, /safe, /history, plus 404 and 500 error handlers. Model loading is handled separately in prediction.py, which caches the loaded model and scaler at import time rather than reloading them on every request.

23. Building HTML Pages

Section prepared by: *Lavanya Thimmapatruni*

All pages extend a single templates/base.html using Jinja2 template inheritance, so the navigation bar, footer and shared stylesheet links only need to be written once:

```

{% extends "base.html" %}
{% block title %}Predict Flood Risk | AquaGuard{% endblock %}

{% block content %}
<form id="flood-predict-form" method="POST" action="{{ url_for('predict') }}">
  <input type="number" name="annual_rainfall" class="form-control form-control-lg">
  ...
</form>
{% endblock %}

```

Page inventory:

- base.html - shared layout, navbar, footer, script/style includes
- home.html - hero section, live model stats, workflow steps, feature cards
- predict.html - the five-field weather reading form with client-side validation
- result.html - probability, risk meter, recommendation and input summary
- chance.html - dedicated red-themed emergency guidance page
- no_chance.html - dedicated green-themed preparedness page
- about.html - methodology, model comparison table, currently deployed model card
- history.html - logged prediction table with running flood/safe counts
- 404.html / 500.html - custom error pages so failures never show a raw stack trace

Styling is hand-written across two files rather than a copied template: static/css/style.css (layout, cards, forms, the red/green theme overrides used by chance.html and no_chance.html) and static/css/animations.css (scroll-reveal and counter animations, isolated so they can be stripped for a reduced-motion pass).

24. Run the Application

| Section prepared by: *Sushma Sajjala*

24.1 Local Development

```
python train_model.py # one-time (or after changing the dataset)
python app.py
# visit http://127.0.0.1:5000
```

24.2 Docker

```
docker build -t flood-prediction-system .
docker run -p 5000:5000 flood-prediction-system
# or, to persist the SQLite history across rebuilds:
docker compose up --build
```

24.3 Render Deployment

render.yaml lets Render provision the service automatically via New + Blueprint:

```
buildCommand: pip install -r requirements.txt && python train_model.py
startCommand: gunicorn --bind 0.0.0.0:$PORT --workers 2 --timeout 120 app:app
```

Render injects the \$PORT environment variable automatically at runtime; config.py already reads it, so no code changes are required between local and hosted environments.

25. Conclusion

| Section prepared by: *Pyla Hasitha (Team Lead)*

This project set out to test whether a small set of routinely collected weather readings could support a useful, self-serve flood risk signal without requiring hydrological simulation infrastructure. Building and fairly benchmarking four different classification algorithms - rather than committing to a single model up front - made the eventual choice of deployed model both transparent and reproducible: anyone can re-run train_model.py and see exactly how each algorithm performed on the same data split.

The measured results (Section 19) show Decision Tree, Random Forest and KNN all reaching 95.65% accuracy with 100% precision and 66.67% recall on the held-out test set, meaning the deployed model has never raised a false flood alarm in testing but does still miss roughly one in three genuine flood years - a limitation worth stating openly rather than hiding behind a single headline accuracy number, especially given how small and imbalanced the source dataset is.

Wrapping the winning model in a Flask application with input validation, a persisted prediction history, and clearly separated alert/safe result pages turns a notebook experiment into something a non-technical user could plausibly operate in the field. The system is intentionally built to be retrained rather than treated as final - dropping a larger, more recent dataset into dataset/flood_dataset.xlsx and re-running the pipeline is enough to update every number this documentation cites, with no code changes required.

Future extensions (live weather API integration, authenticated multi-user access, SMS/email alerting, and tracking multiple model versions side by side) are outlined in the main README.md and in reports/PROJECT_REPORT.md.

Appendix: Team Contribution Summary

Team Member	Role	Sections Authored
Pyla Hasitha	Team Lead	2, 10, 11, 16, 20, 21, 25
Pydi Sri Vaishnavi	Member	1, 5, 7, 14, 18, 22
Lavanya Thimmapatrani	Member	3, 6, 12, 13, 17, 23
Sushma Sajjala	Member	4, 8, 15, 19, 24
Moksitha Burle	Member	9